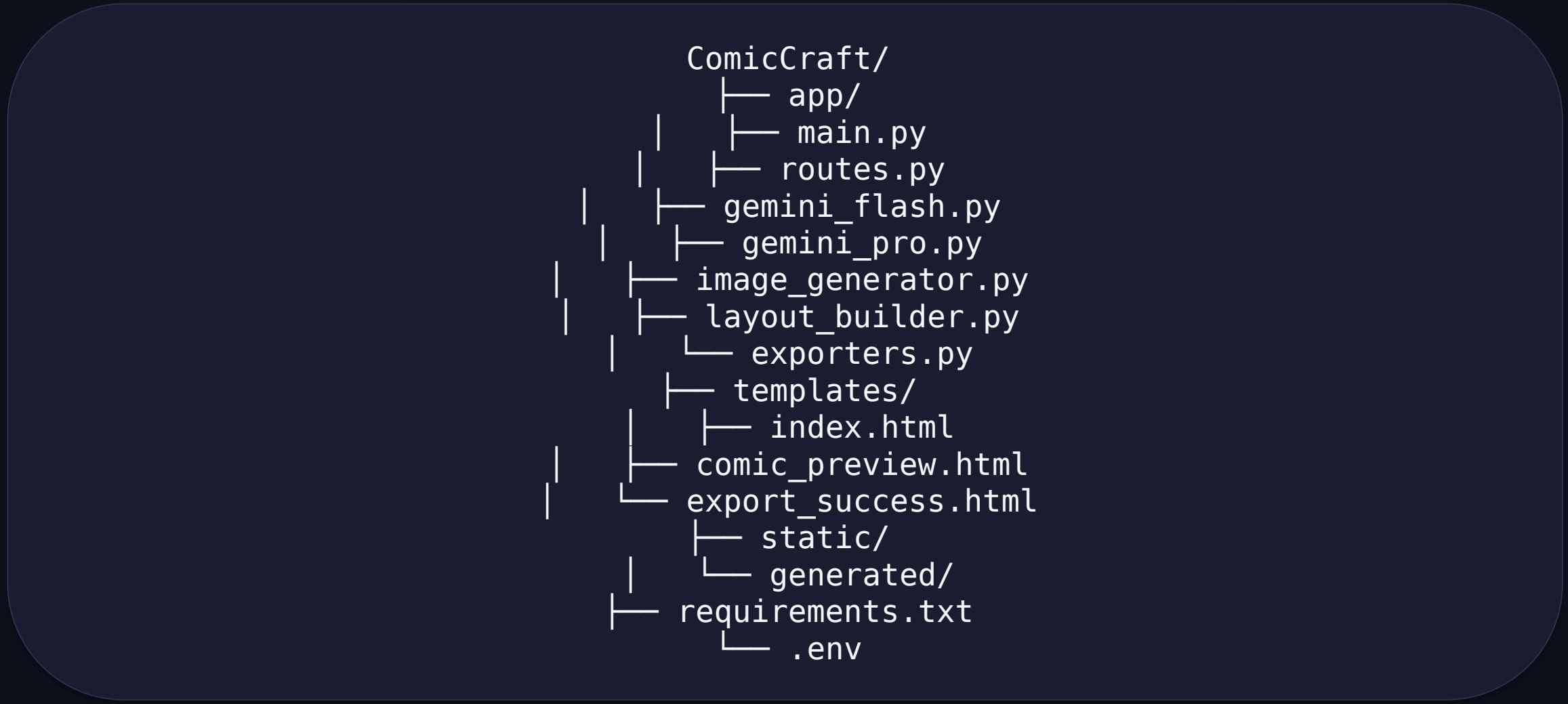
COMICCRAFT

AI Comic Story Creator — Coding

FastAPI + Gemini Flash + Gemini Pro + Stable Diffusion + FPDF

1. Project Structure

Recommended modular layout for the implementation



2. Dependencies & Environment

Install the libraries used by the application

```
# requirements.txt
fastapi
uvicorn
jinja2
google-generativeai
diffusers
transformers
accelerate
torch
fpdf
pillow
python-dotenv

# .env
GEMINI_API_KEY=your_gemini_key
HF_API_KEY=your_huggingface_key
```

3. FastAPI Application

Application entry point and template configuration

```
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from app.routes import router

app = FastAPI(title="ComicCraft AI Comic Creator")

app.mount("/static", StaticFiles(directory="static"), name="static")
templates = Jinja2Templates(directory="templates")

app.include_router(router)

@app.get("/health")
async def health():
return {"status": "ok"}
```

4. Generate a 5-Panel Outline

Gemini Flash handles fast, structured planning

```
import google.generativeai as genai
import os, json

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
model = genai.GenerativeModel("gemini-1.5-flash")

def generate_outline(prompt, character, setting, tone):
    instruction = f"""
    Create a 5-panel comic outline.
    Story: {prompt}
    Character: {character}
    Setting: {setting}
    Tone: {tone}
    Return JSON with title and panels.
    Each panel needs: scene, caption, image_prompt.
    """
    response = model.generate_content(instruction)
    return json.loads(response.text)
```

5. Generate Narration & Dialogue

Gemini Pro expands each outline into richer story content

```
def generate_story(outline):
    model = genai.GenerativeModel("gemini-1.5-pro")
    prompt = f"""
    Expand this comic outline into engaging narration
    and character dialogue.

    Outline:
    {json.dumps(outline)}

    Return one object per panel:
    - narration
    - dialogue
    - characters
    """
    response = model.generate_content(prompt)
    return json.loads(response.text)
```

6. Generate Comic Images

Stable Diffusion converts panel image prompts into illustrations

```
from diffusers import StableDiffusionPipeline
import torch

pipe = StableDiffusionPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5",
    torch_dtype=torch.float16
).to("cuda")

def generate_image(image_prompt, index):
    prompt = (
        f"comic book illustration, {image_prompt}, "
        "clear characters, expressive poses, vibrant panel composition"
    )
    image = pipe(prompt, num_inference_steps=30).images[0]
    path = f"static/generated/panel_{index}.png"
    image.save(path)
    return path
```

7. Build the Comic Layout

Match story data with generated panel images

```
def build_comic_layout(outline, story, image_paths):
                    panels = []

    for i, item in enumerate(outline["panels"]):
                    panels.append({
                      "number": i + 1,
        "title": item.get("title", f"Panel {i+1}"),
                      "scene": item["scene"],
            "caption": item.get("caption", ""),
                 "image": image_paths[i],
            "narration": story[i]["narration"],
             "dialogue": story[i]["dialogue"]
                    })

                    return {
                  "title": outline["title"],
                      "panels": panels
                          }
```

8. FastAPI Routes

Connect the browser form to the complete AI pipeline

```
from fastapi import APIRouter, Request
from fastapi.responses import JSONResponse

router = APIRouter()

@router.get("/")
async def home(request: Request):
return templates.TemplateResponse("index.html", {"request": request})

@router.post("/generate")
async def generate(request: Request):
form = await request.form()

outline = generate_outline(
form["prompt"], form["character"],
form["setting"], form["tone"]
)
story = generate_story(outline)

images = [
generate_image(p["image_prompt"], i + 1)
for i, p in enumerate(outline["panels"])
]

comic = build_comic_layout(outline, story, images)
pdf_path = save_pdf(comic)

return templates.TemplateResponse(
"comic_preview.html",
{"request": request, "comic": comic, "pdf_path": pdf_path}
)
```

9. PDF Export with FPDF

Create a downloadable multi-page comic

```
from fpdf import FPDF

def save_pdf(comic):
    pdf = FPDF()
pdf.set_auto_page_break(auto=True, margin=15)

    for panel in comic["panels"]:
        pdf.add_page()
      pdf.set_font("Arial", "B", 18)
    pdf.cell(0, 10, panel["title"], ln=True)
  pdf.image(panel["image"], x=15, y=25, w=180)

        pdf.ln(115)
      pdf.set_font("Arial", "", 12)
    pdf.multi_cell(0, 8, panel["narration"])
    pdf.multi_cell(0, 8, panel["dialogue"])

  path = "static/generated/comic.pdf"
        pdf.output(path)
        return path
```

10. Frontend Form

Collect prompt, character, setting, tone and art style

```
<form action="/generate" method="post">
  <label>Story Prompt</label>
  <textarea name="prompt" required></textarea>

  <label>Character</label>
  <input name="character" placeholder="Brave fox">

  <label>Setting</label>
  <input name="setting" placeholder="Enchanted forest">

  <label>Tone</label>
  <select name="tone">
    <option>Funny</option>
    <option>Dramatic</option>
    <option>Adventure</option>
  </select>

  <label>Art Style</label>
  <input name="style" placeholder="Comic book">

  <button type="submit">Create Comic</button>
</form>
```

11. End-to-End Pipeline

The core orchestration logic

```
def create_comic(user_input):
    # 1. Structured planning
    outline = generate_outline(
        user_input.prompt,
        user_input.character,
        user_input.setting,
        user_input.tone
    )

    # 2. Creative expansion
    story = generate_story(outline)

    # 3. Visual generation
    image_paths = [
        generate_image(panel["image_prompt"], i + 1)
        for i, panel in enumerate(outline["panels"])
    ]

    # 4. Combine story + images
    comic = build_comic_layout(outline, story, image_paths)

    # 5. Export
    pdf_path = save_pdf(comic)

    return comic, pdf_path
```

12. Run Locally

Start the FastAPI server and open the web application

```
# Create virtual environment
python -m venv venv

# Windows
venv\Scripts\activate

# macOS / Linux
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Start server
uvicorn app.main:app --reload

# Open:
http://127.0.0.1:8000

# FastAPI documentation:
http://127.0.0.1:8000/docs
```

13. Coding Summary

How the modules map to the project requirements

generate_outline() → Gemini Flash → structured 5-panel plan

generate_story() → Gemini Pro → narration and dialogue

generate_image() → Stable Diffusion → comic illustrations

build_comic_layout() → joins text, captions and images

save_pdf() → FPDF → downloadable multi-page comic

FastAPI routes → browser input, generation, preview and export

Jinja2 templates → homepage, comic preview and success page

CODING COMPLETE

ComicCraft connects AI story generation, image generation and PDF export into one FastAPI application.